Manual Técnico - Sistema de Archivos EXT2/EXT3

1. Introducción

El presente manual técnico detalla la estructura interna, la arquitectura y el funcionamiento del sistema de archivos simulado para el Proyecto 2 del curso de Manejo e Implementación de Archivos. Este sistema es capaz de simular discos duros mediante archivos binarios (`.mia`), implementando los sistemas de archivos EXT2 y EXT3 completos con estructuras de inodos, bloques, recuperación de desastres (Journaling) y sistema de permisos (UGO).

2. Arquitectura del Sistema

El sistema se construyó bajo una arquitectura Cliente-Servidor desplegada íntegramente en la nube de Amazon Web Services (AWS).

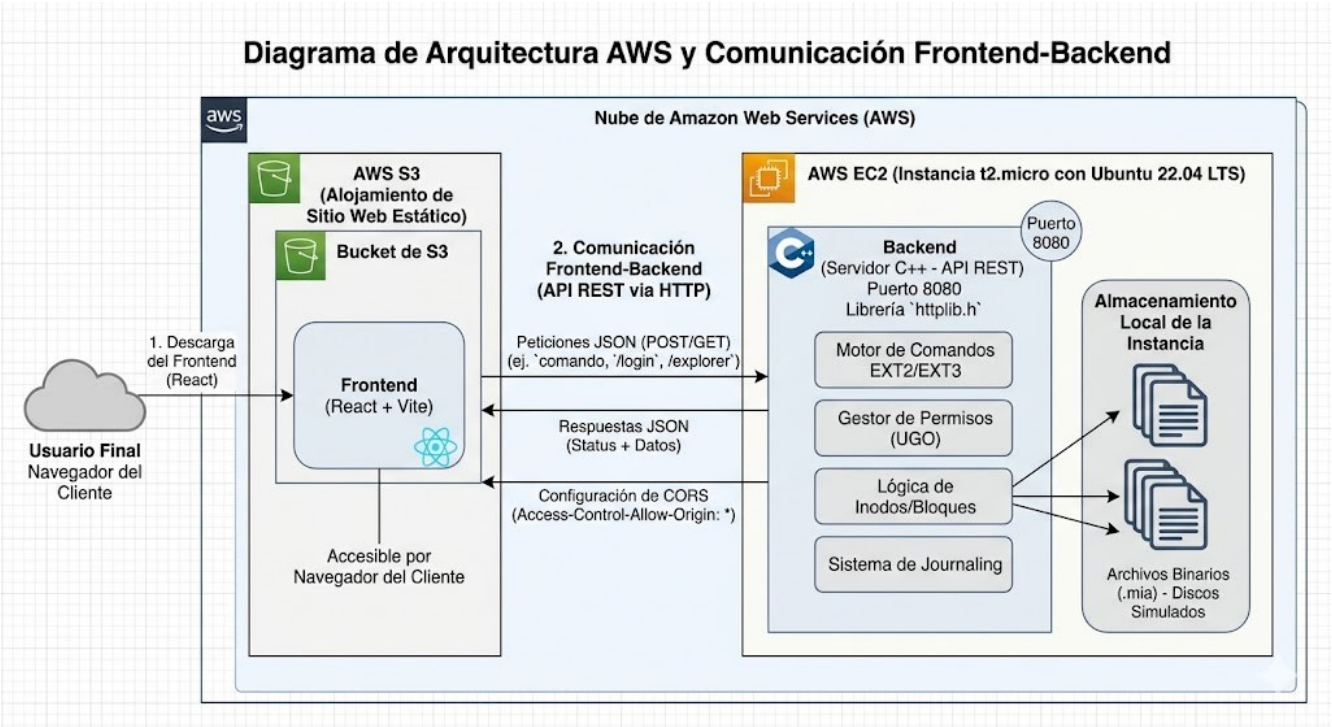
- **Frontend (Cliente):** Aplicación de una sola página (SPA) desarrollada en React (con Vite). Proporciona la Interfaz Gráfica de Usuario (GUI) mediante la cual los usuarios pueden ingresar comandos de consola, visualizar estructuras en un Explorador de Archivos, e iniciar sesión.
- **Backend (Servidor):** Desarrollado en `C++`. Utiliza la librería header-only `httplib.h` para exponer una API REST que escucha en el puerto `8080`. Es el encargado de procesar la lógica pesada: leer y escribir bytes en los archivos binarios, gestionar la memoria de las particiones montadas y aplicar lógica transaccional para EXT3.

2.1 Despliegue en AWS

•**S3 (Frontend):** La compilación de producción de React se aloja en un Bucket de AWS S3 configurado como "Static Website Hosting". Se aplican políticas de Bucket (Bucket Policy) para permitir el acceso de lectura pública.

•**EC2 (Backend):** Instancia `t2.micro` con Ubuntu 22.04 LTS. El backend de C++ se compila y ejecuta de manera persistente (usando `nohup` y `make start`). Las reglas de seguridad (Security Groups) abren el puerto 8080 (TCP) y 22 (SSH).

•**CORS:** El backend en C++ está configurado para responder adecuadamente a las solicitudes de "Preflight" (OPTIONS) con cabeceras `Access-Control-Allow-Origin`, permitiendo que el Frontend en S3 se comunique de forma segura con el EC2.

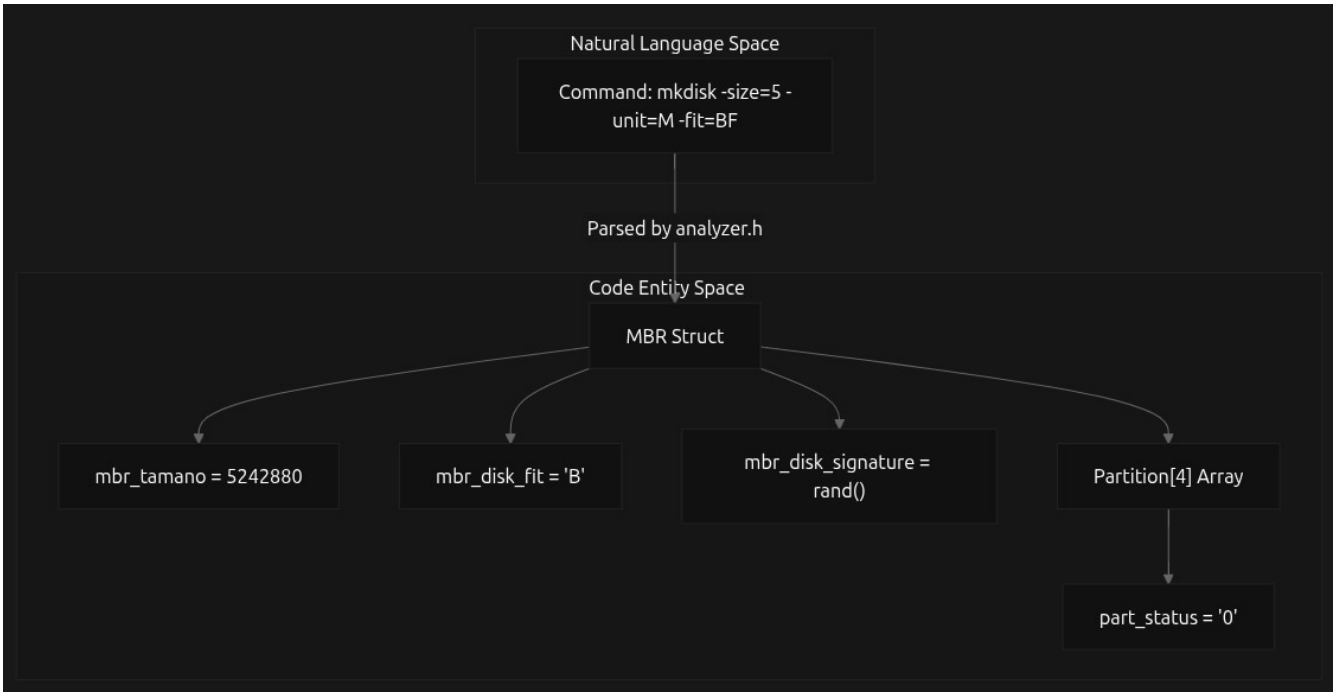


3. Estructuras de Datos

El sistema manipula directamente bytes en archivos binarios, para lo cual se utiliza aritmética de punteros, lectura plana y `structs` definidos en C++.

3.1 Estructuras de Disco

- **MBR (Master Boot Record):** Se sitúa al inicio del disco. Contiene el tamaño total, la fecha de creación, la firma mágica del disco y un arreglo de 4 particiones.
- **EBR (Extended Boot Record):** Utilizado para manejar particiones lógicas. Se implementa como una lista enlazada; cada EBR apunta al siguiente (indicando el byte de inicio del próximo EBR dentro de la partición extendida).



3.2 Estructuras del Sistema de Archivos

•**Superbloque:** Ubicado al inicio de la partición (después del Journaling si es EXT3). Contiene la contabilidad total y libre de inodos y bloques, y la información del número mágico `0xEF53`.

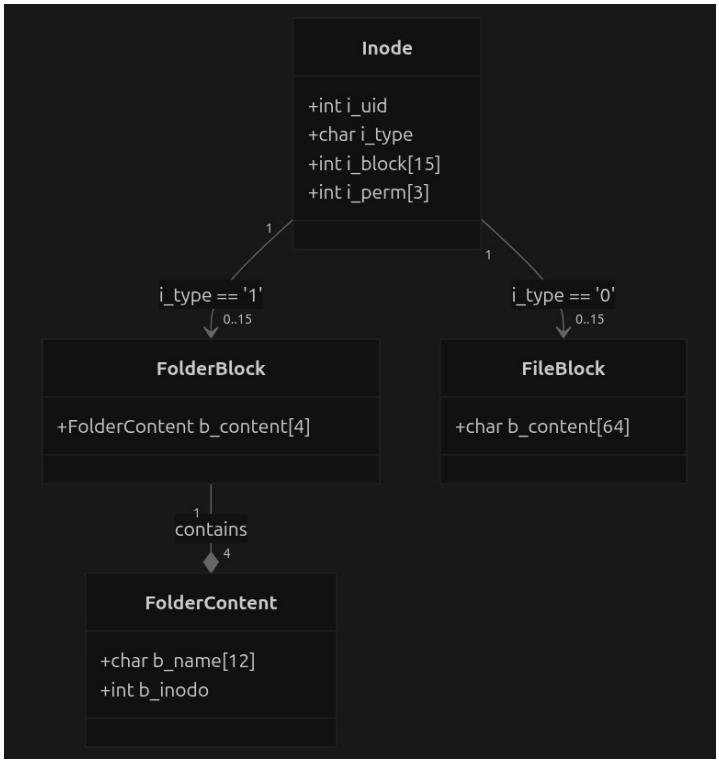
•**Journaling (Solo EXT3):** Un arreglo o bloque contiguo antes del Superbloque destinado a almacenar un historial de operaciones transaccionales (Comando, Tipo, Ruta, Contenido, Fecha). Permite recuperar archivos tras un desastre simulado.

•**Bitmap de Inodos y Bloques:** Representación de 1 byte por elemento que indica con `1` si está ocupado y con `0` si está libre.

•**Inodo (Tabla de Inodos):** Cada inodo representa un archivo o carpeta. Contiene identificadores UGO (Usuario, Grupo), tamaño, fechas (creación, lectura, modificación) y un arreglo de apuntadores a bloques (Directos, e Indirectos Simples, Dobles y Triples).

•**Bloques:**

- •**Carpeta:** Contiene un arreglo de `Content` (Nombre del archivo/carpeta y el ID del inodo al que apunta).
- •**Archivo:** Contiene directamente 64 caracteres (bytes) de información pura.
- •**Apuntadores:** Bloques destinados únicamente a almacenar arreglos de enteros que apuntan a otros bloques (para manejar archivos de gran tamaño).



4. Descripción de Comandos Implementados

4.1 Administración de Discos

•**MKDISK:** Crea un nuevo archivo binario con tamaño pre-asignado llenado de `\0`.

•**RMDISK:** Elimina físicamente el archivo del disco de la ruta especificada.

•**FDISK:** Crea, elimina o redimensiona particiones Primarias, Extendidas o Lógicas modificando el MBR o EBR respectivo.

•**MOUNT:** Carga la información de la partición a una lista en memoria del backend asignándole un ID en el formato `681A` (Carnet, correlativo numérico, letra del disco).

•**UNMOUNT:** Retira la partición de la lista en memoria RAM del servidor.

4.2 Administración de Sistema de Archivos y Usuarios

•**MKFS:** Formatea la partición montada. Crea el Superbloque, bitmaps y estructura del directorio raíz `/`. Inicializa `users.txt` con el usuario Root. Acepta `2fs` (EXT2) o `3fs` (EXT3).

•**LOGIN / LOGOUT:** Verifica credenciales en el archivo `/users.txt` e inicia sesión en el contexto de esa partición para permitir creación de archivos limitados por permisos UGO.

•**MKGRP / RMGRP:** Añade o remueve grupos modificando `/users.txt`.

•**MKUSR / RMUSR:** Añade o elimina usuarios vinculados a grupos preexistentes.

•**CHGRP:** Cambia el grupo primario de un usuario.

4.3 Gestión de Archivos y Directorios

•**MKDIR:** Crea inodos y bloques de carpeta. Si se provee `-p`, crea los directorios padres si no existen.

•**MKFILE:** Crea inodos y bloques de archivo llenando su contenido si se le provee el flag `-size` (con caracteres alfanuméricos sucesivos) o con el `-contenido` de un archivo local externo.

•**CAT:** Concatena y muestra el texto guardado en los bloques de un archivo específico.

•**REMOVE:** Elimina recursivamente un inodo y todos sus bloques y sub-archivos. Limpia los Bitmaps correspondientes.

•**EDIT:** Agrega o reemplaza contenido en un archivo existente.

•**RENAME:** Actualiza el nombre del archivo en el bloque carpeta que lo contiene.

•**COPY / MOVE:** Replican o trasladan el árbol de inodos de un directorio/archivo hacia un nuevo inodo de destino.

•**CHMOD:** Modifica el entero `i_perm` del inodo para cambiar los permisos UGO.

•**CHOWN:** Modifica el `i_uid` para cambiar al propietario del inodo y su contenido.

•**FIND:** Busca coincidencias por nombre exacto o con `*` y `?` desde un directorio de inicio hacia abajo.

4.4 Simulación y Recuperación de Desastres (EXT3)

•**LOSS:** Invalida deliberadamente los Bitmaps y/o Inodos, simulando que el disco se corrompió.

•**RECOVERY:** Lee el bloque de Journaling y vuelve a ejecutar transaccionalmente las operaciones almacenadas para dejar el disco en el estado en que estaba antes de aplicar `LOSS`.

```
# Formatear con EXT3 con journaling
SUCCESS: Partición PartA1 formateada como EXT3 [n=13414 inodos, 40242 bloques]
# Formatear con EXT3 con journaling
SUCCESS: Partición PartA2 formateada como EXT3 [n=17885 inodos, 53655 bloques]
```